Prompt para o GitHub Copilot — Módulo WhatsApp (WAHA) no Osteogest

Como usar este documento

1. Abrir o repositório do **Osteogest** no VS Code.
2. Criar um branch dedicado: `git checkout -b feature/whatsapp-module`.
3. Abrir o **Copilot Chat** em modo **Agent**.
4. Copiar todo o texto entre **INÍCIO DA PROMPT** e **FIM DA PROMPT** e colar no chat.
5. O Copilot começa pela **Fase 0 (análise)** e pára à espera de aprovação. Rever o plano, corrigir o que for preciso e responder "aprovado, avança para a Fase 1".
6. Repetir fase a fase. No fim de cada fase: correr testes/build, rever o diff e fazer commit.

> Recomendação: não aprovar fases seguidas sem testar. Cada fase deve deixar a aplicação a funcionar e com o módulo desligado por omissão.

Pré-requisitos do lado da infraestrutura (ver o documento "Guia de Deploy"):

- Stack `waha` a correr no Portainer, na rede Docker `waha_net`.
- Stack do Osteogest ligada à rede `waha_net`.
- Variáveis `WAHA_API_KEY` e `WAHA_HOOK_HMAC_KEY` definidas nas duas stacks.

INÍCIO DA PROMPT

Papel e objetivo

És um engenheiro sénior de Next.js/TypeScript. Vais criar no Osteogest um **módulo WhatsApp independente** que integra com um servidor **WAHA (WhatsApp HTTP API)** já existente, para enviar mensagens WhatsApp aos pacientes a partir do número da clínica, acompanhar o estado de entrega e receber respostas.

O requisito mais importante é **não causar regressões**: o módulo tem de ser isolado, aditivo, desligado por omissão e removível sem afetar o resto da aplicação.

Stack do Osteogest (confirma no repositório)

- Next.js 16 (App Router; rotas API em `app/api/**/route.ts`; em Next 16 o middleware chama-se `proxy.ts` — confirma qual existe)
- React 19 + Tailwind CSS 4
- TypeScript 5
- Prisma 6 + MySQL 8.4
- Autenticação JWT com `jose` + `bcryptjs`
- Validação com `zod`
- Deploy em Docker/Portainer atrás de nginx-proxy-manager

Regras de isolamento (obrigatórias)

1. **Todo o código novo** fica numa única pasta de módulo: `src/modules/whatsapp/` (ou `modules/whatsapp/`

se o projeto não usar `src/`). Segue a convenção de pastas que encontrares.

2. As pastas `app/` só recebem **ficheiros finos** (route handlers e pages) que importam do módulo e não contêm lógica.
3. **Não alterar** modelos Prisma existentes, tabelas existentes, componentes existentes, testes existentes nem dependências existentes (não atualizar versões).
4. **Sem relações Prisma** para modelos existentes: guardar `patientId`, `appointmentId` e `userId` como colunas simples com índice (sem `@relation`), para não tocar em `Patient`, `Appointment` ou `User`.
5. **Migração apenas aditiva**: só `CREATE TABLE` / `CREATE INDEX` de tabelas com prefixo `whatsapp_`. Mostra-me o SQL gerado antes de aplicar. Proibido `ALTER`/`DROP` em tabelas existentes.
6. **Um único adaptador** (`server/osteogest-adapter.ts`) é o único ficheiro do módulo que conhece os modelos e helpers existentes do Osteogest (pacientes, consultas, utilizadores, autenticação). Se o core mudar, só este ficheiro muda.
7. **Feature flag** `WHATSAPP_MODULE_ENABLED` (default `false`):
   - desligado → todas as rotas do módulo respondem 404, a UI não aparece e nenhum worker arranca;
   - a aplicação tem de funcionar exatamente como hoje com a flag desligada.
8. **Pontos de integração permitidos** em ficheiros existentes (cada alteração mínima, 1–5 linhas, e listada no resumo final):
   - entrada no menu/navegação (visível só com a flag ligada e para quem tiver permissão);
   - slot `<WhatsappPatientPanel patientId={...} />` na ficha do paciente;
   - botão `<WhatsappReminderButton appointmentId={...} />` na consulta/agenda;
   - exclusão de `/api/webhooks/waha` da autenticação JWT em `proxy.ts`/`middleware.ts`;
   - registo do worker em `instrumentation.ts` (criar se não existir);
   - `.env.example` e `docker-compose` (rede externa `waha_net` + variáveis).
     Qualquer outra alteração fora do módulo requer a minha aprovação explícita.
9. Novas dependências: apenas `libphonenumber-js` (e, se necessário, `server-only`). Pede aprovação para qualquer outra.
10. Segue o estilo do projeto (lint, formatação, nomes, padrões de componentes). Não refatores código existente.

Estrutura do módulo

```
src/modules/whatsapp/
  index.ts                    # exports públicos (componentes + tipos) — nada de server-only aqui
  config.ts                   # leitura/validação zod das env vars + isWhatsappEnabled()
  server/
    waha-client.ts            # cliente HTTP do WAHA (server-only)
    waha-types.ts             # schemas zod das respostas/eventos do WAHA
    phone.ts                  # normalização de números -> chatId
    hmac.ts                   # verificação da assinatura dos webhooks
    templates.ts              # render de templates com placeholders
    service.ts                # casos de uso (enviar, estado da sessão, emparelhar, ...)
    webhook-handler.ts        # processamento de eventos do WAHA
    outbox-worker.ts          # fila de envio sequencial + lembretes automáticos
    repository.ts             # acesso Prisma às tabelas whatsapp_*
    osteogest-adapter.ts      # ÚNICO ponto de contacto com o core do Osteogest
    guards.ts                 # requireModuleEnabled(), requireUser(), requireAdmin()
  api/                        # handlers reutilizados pelos route.ts finos
  components/
    WhatsappSettingsPage.tsx
    WhatsappSessionCard.tsx
    WhatsappPairingDialog.tsx
    WhatsappTemplatesEditor.tsx
    WhatsappPatientPanel.tsx
    WhatsappSendDialog.tsx
```

```
    WhatsappMessageHistory.tsx
    WhatsappReminderButton.tsx
    WhatsappBulkRemindersDialog.tsx
  __tests__/
```

Configuração (variáveis de ambiente, server-only)

Variável	Exemplo	Notas
`WHATSAPP_MODULE_ENABLED`	`false`	Feature flag
`WAHA_BASE_URL`	`http://waha:3000`	Rede Docker interna `waha_net`
`WAHA_API_KEY`	(segredo)	Header `X-Api-Key`
`WAHA_WEBHOOK_HMAC_KEY`	(segredo)	Mesmo valor que `WAHA_HOOK_HMAC_KEY` na stack do WAHA
`WAHA_SESSION_NAME`	`osteogest`	Nome da sessão WAHA
`WAHA_TIMEOUT_MS`	`15000`	Timeout de cada pedido

Os segredos **nunca** vão para a base de dados nem para o browser. A UI mostra apenas "API key definida: sim/não". Validar tudo com zod no arranque do módulo; se faltar algo com a flag ligada, registar um erro claro e manter o módulo inativo (sem derrubar a aplicação).

Modelos Prisma (novos, prefixo `whatsapp_`)

Se o projeto usar schema em vários ficheiros (`prisma/schema/`), criar `prisma/schema/whatsapp.prisma`; caso contrário, acrescentar um bloco delimitado no fim de `schema.prisma`. Usar `@@map("whatsapp_...")`.

- **WhatsappSettings** (registo único, id fixo `1`): `clinicPhoneNumber` (dígitos E.164 sem "+", ex.: `351912345678`), `defaultCountry` (default `"PT"`), `sendDelayMs` (default `4000`), `remindersEnabled` (bool, default `false`), `reminderHour` (int, default `18`), `reminderDaysBefore` (int, default `1`), `reminderTemplateKey`, `updatedByUserId`, `updatedAt`.
- **WhatsappTemplate**: `id`, `key` (único), `name`, `body` (text), `active`, `createdAt`, `updatedAt`. Seeds: `appointment_reminder`, `appointment_confirmation`, `free_text`.
- **WhatsappConsent**: `patientId` (único), `granted` (bool), `grantedAt`, `revokedAt`, `userId`, `source` (ex.: `"ficha"`, `"resposta_whatsapp"`).
- **WhatsappMessage**: `id`, `patientId?`, `appointmentId?`, `direction` (`OUTBOUND`|`INBOUND`), `chatId`, `phoneMasked`, `wahaMessageId?` (único), `type` (`TEXT`|`FILE`), `body` (text), `fileName?`, `status` (`QUEUED`|`SENDING`|`PENDING`|`SERVER`|`DEVICE`|`READ`|`PLAYED`|`ERROR`|`RECEIVED`), `error?`, `attempts` (int), `scheduledAt?`, `sentAt?`, `ackAt?`, `sentByUserId?`, `createdAt`, `updatedAt`. Índices em `patientId`, `appointmentId`, `status`, `createdAt`.
- **WhatsappWebhookEvent**: `id`, `requestId` (único; header `X-Webhook-Request-Id`), `event`, `session`, `payload` (Json), `processedAt?`, `error?`, `receivedAt`. Limpar registos com mais de 30 dias.
- **WhatsappSessionState** (registo único): `status`, `meId?`, `pushName?`, `lastEventAt`.

Referência da API do WAHA (confirmada no código do WAHA)

Base: `WAHA_BASE_URL`. Todos os pedidos com `X-Api-Key: <WAHA_API_KEY>` e `Content-Type: application/json`. Sessão: `WAHA_SESSION_NAME` (abaixo `{s}`).

Operação	Pedido	Resposta
Estado da sessão	`GET /api/sessions/{s}`	`{ name, status, me?: { id,`

Operação	Pedido	Resposta
		`pushName } }` — 404 se não existir
Criar sessão	`POST /api/sessions` body `{ "name": "{s}", "start": true, "config": { "metadata": { "app": "osteogest" } } }`	sessão
Iniciar / parar / reiniciar	`POST /api/sessions/{s}/start` · `/stop` · `/restart`	sessão
Terminar sessão (desligar número)	`POST /api/sessions/{s}/logout`	sessão
QR code (PNG)	`GET /api/{s}/auth/qr?format=image` com `Accept: image/png`	imagem PNG
QR code (base64)	`GET /api/{s}/auth/qr?format=image` com `Accept: application/json`	`{ mimetype, data }`
Código de emparelhamento	`POST /api/{s}/auth/request-code` body `{ "phoneNumber": "351912345678" }`	`{ code }` (ex.: `ABCD-EFGH`)
Verificar número	`GET /api/contacts/check-exists?phone=351912345678&session={s}`	`{ numberExists, chatId }`
Enviar texto	`POST /api/sendText` body `{ session, chatId, text, linkPreview: false }`	mensagem com `id`
Enviar ficheiro	`POST /api/sendFile` body `{ session, chatId, caption, file: { mimetype, filename, data: "<base64>" } }` (ou `file: { url }`)	mensagem com `id`
Enviar imagem	`POST /api/sendImage` (mesmo formato de `sendFile`)	mensagem com `id`
LID → telefone	`GET /api/{s}/lids/{lid}` (fazer URL-encode do `@`)	`{ lid, pn }`

Estados da sessão: `STOPPED`, `STARTING`, `SCAN_QR_CODE`, `PASSKEY_REQUIRED`, `PASSKEY_CONFIRMATION_REQUIRED`, `WORKING`, `FAILED`. Só enviar quando `WORKING`.

Formato do chatId: `<dígitos E.164 sem +>@c.us` (ex.: `351912345678@c.us`). Grupos terminam em `@g.us`, estados em `status@broadcast`, canais em `@newsletter`, e contactos podem aparecer como `<n>@lid`.

Erros: 401 (API key), 404 (sessão inexistente), 422/500 (sessão não pronta ou número inválido), timeout. Mapear para erros tipados (`WahaError` com `status`, `code`, `message`) e mensagens amigáveis em português na UI.

Webhooks do WAHA → Osteogest

O WAHA faz `POST` para `http://<serviço-osteogest>:<porta>/api/webhooks/waha` (rede interna) com os eventos `session.status`, `message` e `message.ack`.

Cabeçalhos:

- `X-Webhook-Hmac`: HMAC-SHA512 **em hex** do corpo raw, com a chave `WAHA_WEBHOOK_HMAC_KEY`
- `X-Webhook-Hmac-Algorithm`: `sha512`

- `X-Webhook-Request-Id`: id único do pedido (idempotência)
- `X-Webhook-Timestamp`: timestamp em ms

Corpo: `{ id, timestamp, session, engine, me, environment, event, payload }`

Payloads:

- `session.status` → `{ name, status, statuses: [{ status, timestamp }], data? }`
- `message.ack` → `{ id, from, to, participant, fromMe, ack, ackName }`, com `ack`: `-1` ERROR, `0` PENDING, `1` SERVER, `2` DEVICE, `3` READ, `4` PLAYED
- `message` → `{ id, timestamp, from, fromMe, source, to, body, hasMedia, media?: { url, mimetype, filename } }`

Regras do handler (`POST /api/webhooks/waha`, `runtime = "nodejs"`):

1. Se a flag estiver desligada → 404.
2. Ler o corpo com `await req.text()` **antes** de qualquer parse.
3. Calcular o HMAC e comparar com `crypto.timingSafeEqual` (buffers do mesmo tamanho). Se falhar → 401 e log de aviso (sem o corpo).
4. Se `requestId` já existir em `WhatsappWebhookEvent` → 200 sem reprocessar.
5. Ignorar eventos cuja `session` ≠ `WAHA_SESSION_NAME`.
6. Gravar o evento, processar e responder **200 rapidamente** (o WAHA repete até 10 vezes com backoff exponencial se receber erro). Erros de processamento ficam em `WhatsappWebhookEvent.error`, mas a resposta é 200.
7. `session.status` → atualizar `WhatsappSessionState`; se `FAILED`, `STOPPED` ou `SCAN_QR_CODE`, criar uma notificação para administradores (usar o mecanismo existente através do adaptador; se não existir, mostrar um banner na página de configurações).
8. `message.ack` → atualizar `WhatsappMessage` por `wahaMessageId`; **nunca regredir** o estado (ordem PENDING < SERVER < DEVICE < READ < PLAYED; ERROR é final); gravar `ackAt`.
9. `message` com `fromMe=false`:
   - `from` termina em `@c.us` → número = dígitos antes do `@`;
   - `from` termina em `@lid` → resolver com `GET /api/{s}/lids/{lid}` e usar `pn`;
   - ignorar `@g.us`, `status@broadcast` e `@newsletter`;
   - encontrar o paciente pelo número normalizado (através do adaptador); gravar `WhatsappMessage` INBOUND com status `RECEIVED` (mesmo que o paciente não seja encontrado, com `patientId` nulo);
   - se for resposta a um lembrete enviado nas últimas 72 h: `1`/`sim`/`confirmo` → marcar a consulta como confirmada (via adaptador) e responder com o template de confirmação; `2`/`remarcar`/`não` → sinalizar a consulta para contacto da receção;
   - media recebida: guardar apenas metadados (o WAHA apaga os ficheiros ao fim de 180 s); o download de media fica fora do âmbito da v1.
10. `message` com `fromMe=true` e `source="APP"` (enviadas pelo telemóvel) → gravar como OUTBOUND sem utilizador, para o histórico ficar completo.

Normalização de números (`phone.ts`)

- Usar `libphonenumber-js` com país por omissão `WhatsappSettings.defaultCountry` (`PT`).
- Aceitar `+351 912 345 678`, `00351912345678`, `912345678`, `912-345-678`.
- Devolver `{ e164Digits: "351912345678", chatId: "351912345678@c.us", masked: "351*****678" }` ou um erro de validação.
- Antes do **primeiro** envio a um número, chamar `check-exists`. Se `numberExists=false`, falhar com a mensagem "Este número não tem WhatsApp". Guardar em cache (tabela ou memória, 30 dias) o `chatId` devolvido.

Rotas API (ficheiros finos em `app/api/...` que chamam `src/modules/whatsapp/api/*`)

Todas: `export const runtime = "nodejs"`, `export const dynamic = "force-dynamic"`, `requireModuleEnabled()` (404 se desligado), validação zod do input e erros em JSON `{ error: { code, message } }`.

Método e rota	Permissão	Descrição
`GET /api/whatsapp/settings`	admin	Settings + `{ apiKeyConfigured, hmacConfigured, baseUrl }` (nunca os segredos)
`PUT /api/whatsapp/settings`	admin	Atualiza settings (zod)
`POST /api/whatsapp/settings/test`	admin	`getSession`; devolve alcance, estado e latência
`GET /api/whatsapp/session`	autenticado	Estado + `me` + aviso se `me.id` não corresponder a `clinicPhoneNumber`
`POST /api/whatsapp/session/start`	admin	Cria a sessão se der 404, senão inicia-a
`GET /api/whatsapp/session/qr`	admin	Proxy do PNG, com `Cache-Control: no-store`
`POST /api/whatsapp/session/pairing-code`	admin	Pede código para `clinicPhoneNumber`
`POST /api/whatsapp/session/restart`	admin	Reinicia a sessão
`POST /api/whatsapp/session/logout`	admin	Desliga o número (exige `{ confirm: true }`)
`GET/POST /api/whatsapp/templates`	admin	Listar/criar
`PUT/DELETE /api/whatsapp/templates/[id]`	admin	Editar/desativar (não apagar templates com mensagens)
`GET/PUT /api/whatsapp/consent/[patientId]`	autenticado	Ler/alterar consentimento
`POST /api/whatsapp/messages`	autenticado	Enviar: `{ patientId, appointmentId?, templateKey?, text?, file?: { filename, mimetype, base64 } }`
`GET /api/whatsapp/messages?patientId=&cursor=`	autenticado	Histórico paginado
`POST /api/whatsapp/reminders/preview`	autenticado	Lista de consultas elegíveis para uma data
`POST /api/whatsapp/reminders/enqueue`	autenticado	Enfileira lembretes (sem duplicar para a mesma consulta)
`POST /api/webhooks/waha`	pública (HMAC)	Ver secção de webhooks; excluir da autenticação JWT

Regras de envio (`POST /api/whatsapp/messages`):

- Rejeitar (409) sem consentimento ativo ou se a sessão não estiver `WORKING`.
- Se vier `templateKey`, renderizar com `{{nome}}`, `{{primeiroNome}}`, `{{data}}` (dd/MM/yyyy), `{{hora}}`

(HH:mm), `{{clinica}}`, `{{terapeuta}}`, `{{texto}}` (texto livre escrito pelo utilizador), com fuso `Europe/Lisbon`; placeholders desconhecidos → erro de validação.
- Ficheiros: só PDF, JPEG ou PNG, até 10 MB; validar o mimetype pelos magic bytes.
- Criar a `WhatsappMessage` com status `QUEUED` e deixar o **worker** enviar (nunca enviar em paralelo). Para envios individuais, o worker processa de imediato; a API devolve a mensagem criada e a UI acompanha o estado.
- Rate limit: 20 envios/minuto por utilizador (em memória).

Worker de envio e lembretes (`outbox-worker.ts`)

- Registado em `instrumentation.ts` → `register()`, apenas quando `process.env.NEXT_RUNTIME === "nodejs"` e a flag está ligada.
- Loop a cada 5 s: pega **uma** mensagem `QUEUED` com `scheduledAt <= now` (com lock otimista `UPDATE ... WHERE status='QUEUED'` → `SENDING`), envia, grava `wahaMessageId` e o status `PENDING`/`SERVER`, ou `ERROR` com `error`.
- Entre envios: `sendDelayMs` + jitter aleatório de 0–3 s.
- Retries: até 3 tentativas para timeouts/5xx, com backoff (30 s, 2 min, 10 min); 4xx → `ERROR` imediato.
- Na inicialização, repor mensagens `SENDING` antigas (> 5 min) para `QUEUED`.
- Lembretes automáticos (se `remindersEnabled`): uma vez por dia, a partir de `reminderHour` (Europe/Lisbon), enfileirar lembretes para as consultas de daqui a `reminderDaysBefore` dias, de pacientes com consentimento e ainda sem lembrete. Guardar a última execução para não repetir no mesmo dia.
- Parar o loop de forma limpa em `SIGTERM`.

Interface (React 19 + Tailwind 4, seguindo os componentes existentes)

a) Configurações → Integrações → WhatsApp (admin):

- Cartão de estado com badge: Ligado (verde, `WORKING`), A aguardar emparelhamento (amarelo), Desligado (cinzento), Erro (vermelho); número e nome ligados; aviso se o número ligado for diferente do configurado.
- Botões: Testar ligação · Ligar número · Reiniciar · Terminar sessão (com confirmação).
- Diálogo **Ligar número**, com dois separadores:
  - QR code: imagem de `/api/whatsapp/session/qr`, atualizada a cada 20 s, e polling do estado a cada 3 s até `WORKING`;
  - Código: mostra o código em letra grande, com as instruções "WhatsApp → Definições → Dispositivos ligados → Ligar um dispositivo → Ligar com número de telefone".
- Formulário de settings: número da clínica, país, intervalo entre envios, lembretes automáticos (ligar, hora, dias de antecedência, template).
- Editor de templates com lista de placeholders e pré-visualização com dados fictícios.
- Informação só de leitura: URL do WAHA, "API key definida", "HMAC definido".

b) Ficha do paciente (`<WhatsappPatientPanel />`):

- Interruptor de consentimento, com data e utilizador.
- Botão **Enviar WhatsApp** (desativado, com tooltip explicativo, se não houver consentimento, número válido ou sessão ligada).
- Diálogo de envio: escolha de template ou texto livre, pré-visualização renderizada, anexo opcional (PDF/imagem) e contador de caracteres.
- Histórico em bolhas (enviadas à direita, recebidas à esquerda), com ícones de estado: relógio (QUEUED/

SENDING/PENDING), ✓ (SERVER), ✓✓ (DEVICE), ✓✓ azul (READ/PLAYED), ⚠ (ERROR, com tooltip do erro). Atualização por polling a cada 10 s enquanto o painel estiver visível.

c) Agenda / consultas:

- `<WhatsappReminderButton />`: envia o lembrete de uma consulta e mostra o estado do último lembrete.
- `<WhatsappBulkRemindersDialog />`: escolher a data, ver a lista de elegíveis (com motivo de exclusão: sem consentimento, sem número, já enviado), confirmar e acompanhar o progresso.

Todos os textos da UI em **português de Portugal**.

Segurança e RGPD

- API key e HMAC só no servidor (`import "server-only"` em `server/*`); o browser fala apenas com as rotas do Osteogest.
- Autorização em todas as rotas, reutilizando os helpers de autenticação existentes através do adaptador.
- Validação zod em todos os inputs; limites de tamanho; `Cache-Control: no-store` nas respostas sensíveis.
- **Logs sem conteúdo das mensagens** e com números mascarados (`351*****678`).
- Nos templates, não permitir placeholders com dados clínicos; mostrar um aviso na UI a lembrar que não se devem enviar diagnósticos ou notas clínicas.
- Registar quem enviou cada mensagem (`sentByUserId`) e quem alterou consentimentos.
- Revogação de consentimento: se o paciente responder "STOP"/"parar", revogar o consentimento (`source="resposta_whatsapp"`) e não enviar mais nada.

Testes e qualidade

- Testes unitários (usar o framework já existente no projeto; se não houver, propor Vitest e pedir aprovação) para:
  - `phone.ts` (formatos PT, internacionais, inválidos);
  - `hmac.ts` (assinatura válida, inválida, tamanho diferente, corpo alterado);
  - mapeamento e não-regressão de ack;
  - render de templates (placeholders, fuso horário, placeholders desconhecidos);
  - `webhook-handler` (idempotência, sessão diferente, `@lid`, grupos ignorados, confirmação por "1");
  - `waha-client` com `fetch` mockado (sucesso, 401, 404, timeout).
- No fim de **cada fase** correr: typecheck (`tsc --noEmit`), lint, testes existentes + novos e `next build`. Tudo tem de passar.
- Confirmar explicitamente que, com `WHATSAPP_MODULE_ENABLED=false`, a aplicação se comporta como antes (rotas 404, UI escondida, sem worker).

Plano por fases (pára no fim de cada fase e espera aprovação)

Fase 0 — Análise (sem alterar código):

- Mapear a estrutura do projeto: pastas, `app/`, layout e navegação, helpers de autenticação/JWT e roles, `proxy.ts`/`middleware.ts`, `instrumentation.ts`, o schema Prisma (single ou multi-file), os modelos Patient/Appointment/User (campos de nome, telefone, datas, estado da consulta, terapeuta), o framework de testes e os scripts do `package.json`.
- Entregar: o plano concreto, os nomes reais dos ficheiros a criar, a **lista exata** de pontos de integração em ficheiros existentes (com o diff previsto) e os riscos.

Fase 1 — Fundações: config + flag, `waha-client`, `phone`, `hmac`, modelos Prisma + migração aditiva (mostrar o SQL), seeds de templates, adaptador, guards e testes unitários.

Fase 2 — Sessão e configurações: rotas de settings e de sessão, página de configurações com estado, QR/código e teste de ligação, e entrada no menu.

Fase 3 — Envio: consentimento, rota de envio, worker de envio sequencial, painel do paciente (envio + histórico) e botão de lembrete na consulta.

Fase 4 — Webhooks: rota `/api/webhooks/waha`, exclusão no `proxy.ts`/`middleware.ts`, ack, mensagens recebidas, confirmação/remarcação por resposta e STOP.

Fase 5 — Lembretes automáticos e em massa: agendamento diário, diálogo de envio em massa e limpeza de eventos antigos.

Fase 6 — Entrega: atualizar `.env.example` e o `docker-compose` do Osteogest (serviço da app na rede externa `waha_net` + variáveis), criar `src/modules/whatsapp/README.md` (setup, variáveis, fluxo de emparelhamento, como desligar ou remover o módulo), e um resumo final com todos os ficheiros criados, os ficheiros existentes alterados (com justificação) e as instruções de rollback.

Critérios de aceitação

☐ Com a flag desligada, nada muda na aplicação; todos os testes e o build passam.
☐ Com a flag ligada: um admin liga o número por QR ou código e vê o estado `WORKING`.
☐ Um utilizador envia um template a um paciente com consentimento e vê ✓ → ✓✓ → ✓✓ azul.
☐ O envio é bloqueado sem consentimento, com número inválido ou com a sessão desligada, com mensagens claras.
☐ Um webhook com HMAC inválido recebe 401; um webhook repetido não é processado duas vezes.
☐ A resposta "1" a um lembrete confirma a consulta; "STOP" revoga o consentimento.
☐ Os lembretes em massa são enviados um a um, com intervalo, sem duplicados.
☐ Os logs não contêm texto de mensagens nem números completos.
☐ Remover o módulo = apagar `src/modules/whatsapp/`, os route files finos, os pontos de integração listados e (opcionalmente) as tabelas `whatsapp_*`.

FIM DA PROMPT

Anexo — Referência rápida

Exemplo de envio de texto:

```
POST http://waha:3000/api/sendText
X-Api-Key: <WAHA_API_KEY>
Content-Type: application/json

{
  "session": "osteogest",
  "chatId": "351912345678@c.us",
  "text": "Olá Maria, lembramos a sua consulta amanhã às 10:00. Responda 1 para confirmar ou 2 para
remarcar.",
  "linkPreview": false
}
```

Exemplo de verificação HMAC (Node.js):

```
import { createHmac, timingSafeEqual } from "node:crypto";

export function isValidWahaSignature(rawBody: string, signature: string | null, key: string): boolean {
  if (!signature) return false;
  const expected = createHmac("sha512", key).update(rawBody).digest("hex");
  const a = Buffer.from(expected, "hex");
  const b = Buffer.from(signature, "hex");
```

```
  return a.length === b.length && timingSafeEqual(a, b);
}
```

Exemplo de webhook `message.ack`:

```
{
  "id": "evt_01j...",
  "timestamp": 1790000000000,
  "session": "osteogest",
  "engine": "GOWS",
  "event": "message.ack",
  "payload": {
    "id": "3EB0A1B2C3D4E5F6",
    "from": "351912345678@c.us",
    "to": "351910000000@c.us",
    "fromMe": true,
    "ack": 3,
    "ackName": "READ"
  }
}
```

Templates sugeridos:

- appointment_reminder: "Olá {{primeiroNome}}, lembramos a sua consulta de osteopatia na {{clinica}} no dia {{data}} às {{hora}}. Responda 1 para confirmar ou 2 para remarcar."
- appointment_confirmation: "Obrigado, {{primeiroNome}}! A sua consulta de {{data}} às {{hora}} está confirmada."
- free_text: "{{texto}}"